Assignment No. 4

Name: Aditya Patekar
PRN: 124B1C027

A Dictionary stores keywords & its meanings. Provide facility for adding new keywords, deleting keywords, updating values of any entry. Provide a facility to display whole data sorted in ascending/ Descending order. Also find how many maximum comparisons may require for finding any keyword. Use Height balanced tree and find the complexity for finding a keyword

Code:

```
#include <iostream>
#include <vector>
using namespace std;
class Node {
public:
   string key, meaning;
   Node *left, *right;
   int height;
   Node(string k, string m) {
      key = k;
      meaning = m;
      left = right = NULL;
      height = 1;
   }
};
class AVL {
   Node* root;
   int getHeight(Node* n) {
      if (!n) return 0;
      return n->height;
   }
   int getBalance(Node* n) {
      if (!n) return 0;
      return getHeight(n->left) - getHeight(n->right);
   }
   Node* rightRotate(Node* y) {
      Node* x = y->left;
      Node* t = x->right;
      x->right = y;
      y->left = t;
      y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
      x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
```

```
    return x;
}
Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* t = y->left;
    y->left = x;
    x->right = t;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    return y;
}
Node* insert(Node* node, string key, string meaning) {
    if (!node) return new Node(key, meaning);
    if (key < node->key)
        node->left = insert(node->left, key, meaning);
    else if (key > node->key)
        node->right = insert(node->right, key, meaning);
    else {
        node->meaning = meaning;
        return node;
    }
    node->height = 1 + max(getHeight(node->left), getHeight(node->right));
    int balance = getBalance(node);
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}
Node* minValue(Node* node) {
    Node* curr = node;
    while (curr->left) curr = curr->left;
    return curr;
}
Node* deleteNode(Node* root, string key) {
    if (!root) return root;
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
```

```
    else {
        if (!root->left || !root->right) {
            Node* temp = root->left ? root->left : root->right;
            if (!temp) {
                temp = root;
                root = NULL;
            } else
                *root = *temp;
            delete temp;
        } else {
            Node* temp = minValue(root->right);
            root->key = temp->key;
            root->meaning = temp->meaning;
            root->right = deleteNode(root->right, temp->key);
        }
    }
    if (!root) return root;
    root->height = 1 + max(getHeight(root->left), getHeight(root->right));
    int balance = getBalance(root);
    if (balance > 1 && getBalance(root->left) >= 0)
        return rightRotate(root);
    if (balance > 1 && getBalance(root->left) < 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if (balance < -1 && getBalance(root->right) <= 0)
        return leftRotate(root);
    if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}
Node* search(Node* root, string key, int &comp) {
    if (!root) return NULL;
    comp++;
    if (key == root->key) return root;
    if (key < root->key)
        return search(root->left, key, comp);
    else
        return search(root->right, key, comp);
}
void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->key << " : " << root->meaning << endl;
    inorder(root->right);
}
```

```
    void reverseInorder(Node* root) {
        if (!root) return;
        reverseInorder(root->right);
        cout << root->key << " : " << root->meaning << endl;
        reverseInorder(root->left);
    }
public:
    AVL() { root = NULL; }
    void add() {
        string k, m;
        cout << "Enter key and meaning: ";
        cin >> k >> m;
        root = insert(root, k, m);
    }
    void remove() {
        string k;
        cout << "Enter key to delete: ";
        cin >> k;
        root = deleteNode(root, k);
    }
    void update() {
        string k, m;
        cout << "Enter key to update: ";
        cin >> k;
        int comp = 0;
        Node* res = search(root, k, comp);
        if (res) {
            cout << "Enter new meaning: ";
            cin >> m;
            res->meaning = m;
        } else {
            cout << "Key not found\n";
        }
    }
    void displayAsc() {
        inorder(root);
    }
    void displayDesc() {
        reverseInorder(root);
    }
    void findKey() {
        string k;
        cout << "Enter key to search: ";
        cin >> k;
        int comp = 0;
        Node* res = search(root, k, comp);
        if (res)
            cout << "Found with comparisons: " << comp << endl;
```

```
        else
            cout << "Not found. Comparisons: " << comp << endl;
    }
    void maxComparisons() {
        cout << "Max comparisons (height): " << getHeight(root) << endl;
    }
};
int main() {
    AVL obj;
    int ch;
    do {
        cout << "\n1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit\n";
        cin >> ch;
        if (ch == 1) obj.add();
        else if (ch == 2) obj.remove();
        else if (ch == 3) obj.update();
        else if (ch == 4) obj.displayAsc();
        else if (ch == 5) obj.displayDesc();
        else if (ch == 6) obj.findKey();
        else if (ch == 7) obj.maxComparisons();
    } while (ch != 0);
    return 0;
}
```

Output:

```
1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
1
Enter key and meaning: Mango Fruit

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
1
Enter key and meaning: Cat Animal

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
Mumba□ □□ □□ □□ □□ □1
Enter key and meaning: Mumbai City

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
4
Cat : Animal
Mango : Fruit
Mumbai : City

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
2
Enter key to delete: Mango

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
5
Mumbai : City
Cat : Animal

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
6
Enter key to search: Cat
Found with comparisons: 2

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
3
Enter key to update: Mumbai
Enter new meaning: Pune

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
7
Max comparisons (height): 2

1.Add 2.Delete 3.Update 4.Display ASC 5.Display DESC 6.Search 7.MaxComp 0.Exit
0
```